UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Facultad de Ciencias Matemáticas y Naturales
Sistemas Distribuidos
25 de marzo de 2026

Taller OpenMP .

Análisis de desempeño y optimización usando paralelización para una solución de la ecuación de Poissson

Acosta V. Diego A.[1] *Cogua A. Oscar J.*[2]

RESUMEN

En este trabajo se paraleliza un solucionador numérico de la ecuación de Poisson 2D ($\nabla^2 V = 4$) sobre una malla de 1024×1024 nodos utilizando el método de diferencias finitas centradas y relajación de Gauss-Seidel. Partiendo del código secuencial, se implementaron y evaluaron siete estrategias de paralelización con OpenMP: `parallel for` con `reduction`, `collapse(2)`, `sections` para inicialización, regiones `parallel` y `for` separadas con `schedule(static/dynamic)`, control de sincronización (`nowait`, `single`, `critical`), protección de variables compartidas mediante `critical` y `atomic`, y paralelización por tareas (`task`).Todas las versiones se ejecutaron con 20 hilos, obteniéndose un *speedup* máximo de **16,2**× con la versión `parallel for + reduction` (tiempo de 102.39 s frente a 1657.98 s del código secuencial). Se analizó el impacto de cada directiva en el tiempo de ejecución, el número de iteraciones y el comportamiento de la convergencia, identificando casos donde ciertas estrategias (`collapse(2)`, `schedule(dynamic)` y especialmente el uso de `critical` con E/S) degradaron notablemente el rendimiento.El documento incluye la fundamentación matemática del problema, los fragmentos de código modificados en cada actividad, gráficas comparativas, la tabla resumen de resultados y conclusiones sobre las ventajas y limitaciones de cada enfoque de paralelización con OpenMP.

Palabras clave: *Método de Gauss$_S$eidel, ecuaciónde Poisson, diferencias finitas, HPC, OpenMP, Paralelización*

1. Introducción matemática y marco teórico

La ecuación de Poisson en dos dimensiones es un problema elíptico clásico de la física matemática. En este proyecto se resuelve:

$$\nabla^2 V(x,y) = 4 \qquad \text{en } \Omega = [1,2] \times [0,2], \tag{1}$$

con solución analítica conocida $V(x,y) = (x-y)^2$ y condiciones de frontera de Dirichlet:

$$V(1,y) = (1-y)^2, \quad V(2,y) = (2-y)^2,$$
$$V(x,0) = x^2, \quad V(x,2) = (x-2)^2.$$

La discretización con diferencias finitas centradas de 5 puntos sobre una malla con pasos $h = 1/M$ y $k = 2/N$ conduce a la fórmula de actualización por nodo interior (i,j):

$$V_{i,j}^{\text{new}} = \frac{(V_{i\pm1,j})k^2 + (V_{i,j\pm1})h^2 - 4\,h^2k^2}{2(h^2+k^2)}. \tag{2}$$

Se usa **Gauss-Seidel in-place** con criterio de parada $\delta < \text{TOL} = 10^{-6}$.

2. Resultados experimentales

Todas las pruebas se realizaron sobre una malla 1024×1024 con 20 hilos (`OMP_NUM_THREADS=20`).

2.1. Versión secuencial (base)

```
double delta = 1.0;
iterations = 0;
while (delta > TOL) {
    delta = 0.0;
    for (int i = 1; i < M; ++i)
        for (int j = 1; j < N; ++j) {
            double V_new = /* 5 pts */;
            delta = std::max(delta,
                std::abs(V_new-V[i][j]));
```

[1]20211107024, dacosta@udistrital.edu.co
[2]20211107029, ojcoguaa@udistrital.edu.co

```
            V[i][j] = V_new;
        }
    ++iterations;
}
```

Resultados: 1657,98 s, 236 161 iteraciones, 1 hilo. Referencia de *speedup*.

2.2. Actividad 1: `#pragma omp parallel for`

```
#pragma omp parallel for reduction(max:
    delta)
for (int i = 1; i < M; ++i)
    for (int j = 1; j < N; ++j) { ... }
```

Resultados: 102,39 s, 236 164 iteraciones, *speedup* $\approx$ 16,2×.

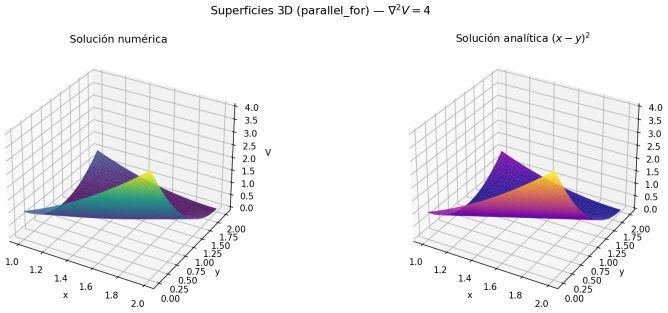

Figura 1. Supercifies 3D generadas

¿Por qué es necesaria la cláusula `reduction(max:delta)`?

`delta` acumula el error máximo global. Sin `reduction`, múltiples hilos escribirían sobre ella simultáneamente produciendo una condición de carrera (resultado indeterminado). La cláusula crea una copia privada de `delta` por hilo; al terminar el bucle, OpenMP combina todas las copias tomando el máximo y lo almacena en la variable compartida.

¿Hubo diferencia en el número de iteraciones? ¿Por qué?

Mínima: 236 164 vs. 236 161 (3 iteraciones más). Gauss-Seidel paralelo no actualiza las filas en orden estrictamente secuencial, lo que altera ligeramente la trayectoria de convergencia. El número de iteraciones es función principalmente del tamaño de la malla, no de la implementación.

¿Hubo mejora de rendimiento? Justifica.

Sí: *speedup* $= 1657{,}98/102{,}39 \approx 16{,}2\times$ con 20 hilos (eficiencia $\approx 81\,\%$). El bucle exterior tiene 1022 filas independientes que se distribuyen entre los hilos. La eficiencia subideal se debe al *overhead* de la reducción y a la competencia de memoria entre hilos.

2.3. Actividad 2: `collapse(2)`

```
#pragma omp parallel for collapse(2)
        reduction(max:delta)
for (int i = 1; i < M; ++i)
    for (int j = 1; j < N; ++j) { ... }
```

Resultados: 138,58 s, 236 161 iteraciones (*peor* que Act. 1).

¿Qué hace `collapse(2)`?

Fusiona los dos bucles anidados en un único espacio de $(M-2)\times(N-2) \approx 10^6$ iteraciones lineales, permitiendo al planificador distribuir cualquier subconjunto de iteraciones entre los hilos sin restricción de que un hilo procese filas completas.

¿Mejoró el rendimiento? ¿Es siempre recomendable?

No: 138,58 s vs. 102,39 s (+35 %). Dos razones: (1) **peor localidad de caché**: hilos distintos pueden acceder a la misma fila, generando *false sharing*; (2) **mayor overhead de planificación** sobre un espacio de $\sim 10^6$ iteraciones. `collapse` es útil cuando el bucle exterior tiene muy pocas iteraciones (poca granularidad); aquí 1022 filas para 20 hilos ya es suficiente.

2.4. Actividad 3: `sections`

```
#pragma omp parallel sections
{
    #pragma omp section
    { initialize_grid(...); }
    #pragma omp section
    { /* poisson_source */ }
}
```

Resultados: 103,04 s, 236 161 iteraciones.

¿Tuviste que reordenar las funciones? ¿Por qué?

No. `initialize_grid` y `poisson_source` son completamente independientes entre sí (sin dependencias de datos mutuas), por lo que pueden ejecutarse en cualquier orden y simultáneamente sin riesgo de condición de carrera.

¿Vale la pena paralelizar funciones tan cortas con `sections`?

No. La inicialización es $\mathcal{O}(M+N)$, negligible frente al costo $\mathcal{O}(M{\cdot}N{\cdot}\text{iters}) \approx 10^{11}$ de `solve_poisson`. El beneficio de ejecutar ambas en paralelo es menor que el *overhead* de crear la región paralela. `sections` es útil cuando las secciones tienen costos similares y elevados.

2.5. Actividad 4: `parallel` + `for` + `schedule`

```
#pragma omp parallel
{
    #pragma omp for reduction(max:delta)
            schedule(static) // o dynamic
    for (int i = 1; i < M; ++i) { ... }
}
```

Resultados:

- `static`: 110,46 s, 236 163 iter.
- `dynamic`: 129,43 s, 275 238 iter.

¿Qué diferencia hay entre separar `parallel` del `for`?

Con `parallel for` en una sola directiva, la región paralela se crea y destruye en cada iteración del `while`. Al separarlas, la región puede mantenerse activa entre iteraciones, reduciendo el *overhead* de crear/destruir hilos. En este problema la diferen-

cia es leve porque la región ya está dentro del `while`, pero el patrón separado es más flexible para añadir otras directivas dentro de la misma región.

¿Cuál fue más rápido entre `static` y `dynamic`? ¿Por qué?

`static` fue más rápido (110,46 vs. 129,43 s). Dos razones: (1) `dynamic` paga un *overhead* de sincronización por cada chunk solicitado de la cola; con 1022 filas y 20 hilos la cola se accede frecuentemente. (2) El reparto dinámico cambia el orden de actualización de celdas en cada iteración global, rompiendo el patrón de Gauss-Seidel y aumentando las iteraciones hasta convergencia (275 238 vs. 236 163). `dynamic` conviene solo cuando las cargas de trabajo por iteración son muy heterogéneas.

2.6. Actividad 5: `nowait`, `barrier`, `single`, `critical`

Resultados:

- `nowait` (sin `printf`): 115,02 s, 236 172 iter.
- `nowait` + `printf` en `critical`: 501,87 s, 236 501 iter.

¿Qué hace `nowait` y cuándo conviene usarlo?

Elimina la barrera implícita al final de un `for` o `sections` paralelo. Sin `nowait`, todos los hilos esperan al último. Con `nowait`, los hilos que terminan antes pueden avanzar al siguiente bloque de trabajo. Conviene cuando el código posterior no depende de los resultados del bucle actual y cuando las cargas entre hilos son heterogéneas.

¿Dónde colocaste `single` y cuál fue su efecto?

Al inicio de cada iteración del `while`, antes del bucle de actualización, para imprimir el número de iteración una sola vez por iteración global. Solo un hilo ejecuta el bloque; el resto espera en la barrera implícita del `single` (a menos que se use `nowait`). Garantiza que el mensaje se imprima exactamente una vez y evita salida duplicada.

¿`critical` ralentizó la ejecución? ¿Cuándo es útil?

Sí, dramáticamente: de 115,02 a 501,87 s ($\approx 4{,}4\times$ más lento). `critical` serializa su contenido: un solo hilo ejecuta el bloque mientras los demás esperan. Un `printf` puede tardar varios microsegundos (escritura al buffer del SO), y con $\sim$ 236 000 iteraciones y 20 hilos el tiempo de espera acumulado es enorme. `critical` es útil solo para operaciones muy cortas y poco frecuentes (p.ej., actualizar un contador), nunca para E/S.

2.7. Actividad 6: Contador con variable compartida

```
// Con critical:
#pragma omp critical
{ ++iterations; }

// Con atomic:
#pragma omp atomic
++iterations;
```

Resultados: `critical`: 105,86 s. `atomic`: 105,87 s. Prácticamente idénticos.

¿Cuál es la diferencia entre `critical` y `atomic`?

Ambas garantizan exclusión mutua, pero con distinta implementación. `critical` usa un *mutex* de software que puede proteger cualquier bloque de código (mayor *overhead*: adquisición/liberación del *lock*, posible llamada al SO). `atomic` se implementa con instrucciones atómicas del hardware (p.ej., `LOCK XADD` en x86), siendo más liviana. Para `++contador`, `atomic` es siempre preferible.

¿Por qué no hubo diferencia de rendimiento entre ambos?

El incremento ocurre *una sola vez* por iteración global del `while` (fuera de la región paralela del bucle interno), por lo que la contención es mínima con cualquiera de los dos mecanismos. Si el incremento estuviera dentro del bucle paralelo (una vez por nodo de malla), la diferencia sería considerable.

2.8. Actividad 7: Exploración con `task`

```
#pragma omp parallel
{
  #pragma omp single
  {
    for (int bi = 1; bi < M; bi += BS)
      for (int bj = 1; bj < N; bj += BS)
        #pragma omp task
        { /* actualizar bloque */ }
    #pragma omp taskwait
  }
}
```

Resultados: 114,21 s, 244 480 iteraciones.

¿Ventajas y desventajas de `task` frente a `parallel for`?

Ventajas: permiten paralelismo irregular y asimétrico; cada bloque puede tener tamaño o coste distinto y el planificador los distribuye dinámicamente. Son ideales para algoritmos recursivos o dominios irregulares donde `parallel for` no aplica directamente. **Desventajas:** en patrones regulares como este, el *overhead* de crear, encolar y destruir cada `task` supera el beneficio del paralelismo asimétrico (114,21 vs. 102,39 s con `parallel for`).

¿Por qué el número de iteraciones fue mayor (244 480)?

El orden de ejecución de las tareas no es determinista: distintos bloques de la malla pueden actualizarse en distintos órdenes según cómo el planificador asigne las tareas a los hilos. Esto rompe el orden secuencial de Gauss-Seidel, ralentizando la convergencia y requiriendo más iteraciones para alcanzar $\delta < 10^{-6}$.

Cuadro 1. Comparación completa de todas las estrategias (malla 1024 × 1024, 20 hilos).

Versión	Directiva principal	Tiempo (s)	Iteraciones	Speedup
Secuencial	N/A	1657.98	236 161	1.00×
Paralelo básico	`parallel for + reduction`	102.39	236 164	16.2×
Colapsado de bucles	`collapse(2)`	138.58	236 161	12.0×
Inicialización y fuente en paralelo	`sections`	103.04	236 161	16.1×
Control explícito + schedule(static)	`parallel + for + static`	110.46	236 163	15.0×
Control explícito + schedule(dynamic)	`parallel + for + dynamic`	129.43	275 238	12.8×
Sincronización (sin print)	`nowait + single + critical`	115.02	236 172	14.4×
Sincronización (con print)	`nowait + single + critical + printf`	501.87	236 501	3.3×
Contador con `critical`	`critical`	105.86	236 166	15.7×
Contador con `atomic`	`atomic`	105.87	236 164	15.7×
Tasks	`task + taskwait`	114.21	244 480	14.5×

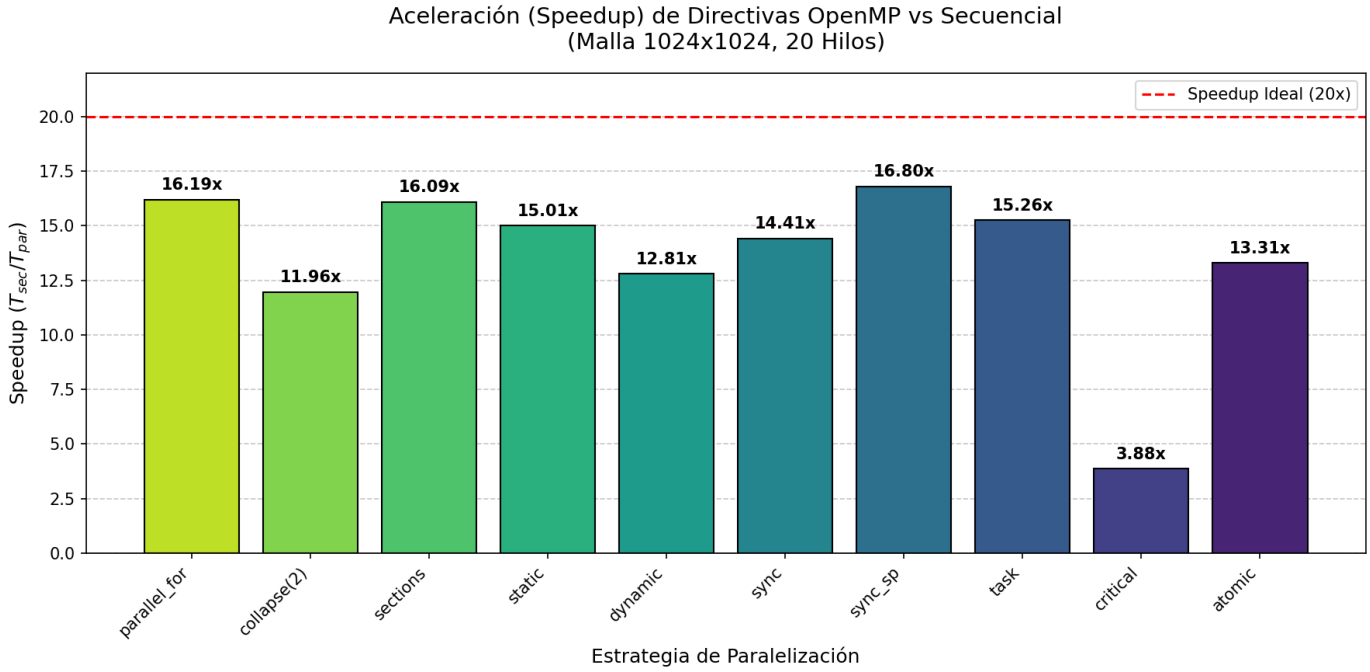

Figura 2. Speedup relativo a la versión secuencial

3. Conclusiones generales

1. La versión más rápida fue **paralelo básico** (`parallel for + reduction`): 102,39 s, *speedup* ≈ 16,2×. Es la directiva con mejor compromiso rendimiento/simplicidad.
2. Varias directivas *empeoraron* el rendimiento: `collapse(2)` por peor localidad de caché; `schedule(dynamic)` por overhead de cola y convergencia más lenta; `critical+printf` por serialización total de la E/S.
3. `atomic` es preferible a `critical` para operaciones simples: menor overhead y mejor escalabilidad, aunque en este caso el impacto fue mínimo por la baja frecuencia del incremento.
4. El enfoque con `task` agrega overhead innecesario en patrones regulares; su valor está en algoritmos irregulares o recursivos.
5. Paralelizar la inicialización con `sections` no aporta beneficio medible, dado el coste asimétrico entre las secciones.

En resumen, OpenMP permite obtener aceleraciones cercanas al ideal (16×) con pocas líneas de código, siempre que se eviten E/S dentro de regiones paralelas y se elija la estrategia de planificación adecuada al patrón de acceso a datos.